

Quiz-1

Total points 15/25 ?

Instructions to Students:

- Write your **FULL NAME IN CAPITAL LETTERS** at the top of the answer sheet.
- Write your **ROLL NUMBER IN CAPITAL LETTERS** clearly next to your name.
- Read each question carefully before answering.
- Attempt **all questions**. Each question may be **MCQ (one correct)** or **MSQ (multiple correct)**.
- For code-based questions, carefully analyze the given CUDA code snippets before answering.
- Mark your choice(s) clearly (e.g., Q1: a, Q2: b,c).
- **Do not open other tabs, notes, or external resources** while giving the exam.
- **Do not use mobile phones, calculators, or any external device** to assist with the quiz.
- **Do not leave your seat** unless permitted by the invigilator.
- **Do not share answers or discuss** with peers during the exam.

Important Notes

- Time allowed: **30 minutes only**.
- All responses must be submitted before the time limit – **late submissions will not be considered**.
- This is a **closed-book, closed-internet** exam.
- Each question carries equal marks unless otherwise specified.

The respondent's email (**cs25mtech11015@iith.ac.in**) was recorded on submission of this form.

0 of 0 points

Write your roll number in **CAPITAL** Letters *

CS25MTECH11015

Write your name in **CAPITAL** Letters *

LOCHAN SURYA TEJA NEELI



Below You can find your questions.

15 of 25 points

When a warp diverges, the hardware reconverges threads:

0/1

- ☐ At the end of the kernel.
- ☒ At the earliest post-dominator (merge point) of the branch.
- ☒ At the function return point.
- ☐ Only after __syncthreads().

Correct answer

- ☒ At the earliest post-dominator (merge point) of the branch.

Select all correct statements about CUDA blocks:

1/1

- ☐ Blocks can communicate using shared memory.
- ☒ Block size is limited by hardware (typically up to 1024 threads).
- ☒ Each block can have 1D, 2D, or 3D configuration.
- ☐ Blocks can directly share registers.



```
__global__ void k() {  
    __shared__ unsigned s;  
    if (threadIdx.x==0) s=0;  
    if (threadIdx.x==1) s+=1;  
    if (threadIdx.x==100) s+=2;  
    if (threadIdx.x==0) printf("s=%u\n", s);  
}
```

1/1

Launched as `k<<<1,1024>>>()`. What is guaranteed?

- ☐ Always prints s=3
- ☒ Data race unless a barrier is used
- ☐ Deterministic since warp 0 runs first
- ☐ None

For shared memory bank conflicts, consider an array access pattern where thread `t` accesses

0/1

`shared_array[t * stride + offset]`.

Given 32 banks and 1024 threads per block, which configuration causes the worst-case bank conflict?

- ☒ stride = 16, offset = 0
- ☐ stride = 8, offset = 1
- ☐ stride = 33, offset = 0
- ☐ stride = 1, offset = 16

Correct answer

- ☒ stride = 33, offset = 0



A kernel is launched with `<<<dim3(4,2,1), dim3(8,16,2)>>>`. How many total threads are launched? 1/1

- ☒ 2048
- ☐ 8192
- ☐ 1024
- ☐ 4096

Warp size = 32. If 1000 threads are launched in a kernel, how many full warps and partially filled warps are created? 1/1

- ☐ 30 full, 2 partial
- ☒ 31 full, 1 partial
- ☐ 33 full, 1 partial
- ☐ 32 full, 0 partial

Choose all correct statements about `cudaDeviceSynchronize()`: 1/1

- ☒ It blocks the CPU until GPU completes execution of previously launched kernels.
- ☐ It synchronizes threads inside a block.
- ☐ It is required after every kernel call.
- ☒ It ensures all previously launched kernels are finished.



For a warp with threads 0–31, executing:

1/1

```
if (threadIdx.x < 8) do A;  
else if (threadIdx.x < 16) do B;  
else do C;
```

How many distinct divergent paths are there ?

- ☐ 8
- ☐ 2
- ☒ 3
- ☐ 1

Consider a warp executing a divergent if–else branch. If half of the threads evaluate the if path and the other half evaluate the else path, the warp execution efficiency (i.e., the fraction of active threads per issued instruction) is reduced to:

1/1

- ☒ 50%
- ☐ 25%
- ☐ 0%
- ☐ 75%

Which CUDA memory(s) is/are shared among all the threads in across all blocks ?

1/1

- ☐ Shared Memory
- ☒ Global Memory
- ☐ Registers
- ☒ L2 Cache



Which are consequences of high register usage per thread?

0/1

- ☐ Increased spilling into global memory.
- ☒ Lower occupancy because fewer threads fit on an SM.
- ☒ Fewer resident blocks per SM.
- ☐ Higher performance always, because registers are fastest.

Correct answer

- ☒ Lower occupancy because fewer threads fit on an SM.
- ☒ Increased spilling into global memory.
- ☒ Fewer resident blocks per SM.

If each thread needs 80 registers, SM has 64K registers, what happens if 1024 threads per block are launched? 0/1

- ☐ Spilling occurs to global memory.
- ☐ Launch fails with runtime error.
- ☒ Spilling occurs to shared memory.
- ☐ All threads fit, full occupancy.

Correct answer

- ☒ Spilling occurs to global memory.



```
__global__ void k()
{
    printf("kernel");
}
int main(){
    k<<<G,B>>>();
    printf("host\n");
    cudaDeviceSynchronize();
}
```

0/1

What is/are the possible output(s) ?

- ☒ Gives undefined behaviour unless cudaDeviceSynchronize() precedes host printf
- ☒ Host print may appear before kernel prints
- ☐ Always kernel prints before host
- ☐ Always host prints before the kernel prints

Correct answer

- ☒ Host print may appear before kernel prints

Which types of workloads typically run better on CPUs than GPUs?

1/1

- ☒ Branch-intensive tasks with irregular memory access.
- ☐ Dense linear algebra (e.g., GEMM).
- ☒ OS kernel execution and system calls.
- ☐ Image convolution and FFTs.



Consider a kernel where each thread computes

0/1

$a[\text{threadIdx.x}] = \text{threadIdx.x} * \text{threadIdx.x};$

If launched with 4 blocks of 128 threads, what is the maximum index written?

- ☐ 127
- ☐ 255
- ☒ 511
- ☐ 1023

Correct answer

- ☒ 127

A warp in modern NVIDIA GPUs typically consists of:

1/1

- ☐ 8 threads
- ☐ 16 threads
- ☒ 32 threads
- ☐ 64 threads

In CUDA, a kernel function is executed:

0/1

- ☐ Only by the CPU
- ☒ By multiple threads on the GPU in parallel
- ☒ Only once per program launch
- ☐ Sequentially by each GPU core

Correct answer

- ☒ By multiple threads on the GPU in parallel



A warp of 32 threads accesses shared memory addresses with stride = 2 (i.e., thread $i \rightarrow s[2*i]$). Assuming 32 banks, what happens?

- ☒ Bank conflicts occur because stride is not 1.
- ☐ Broadcast occurs, all threads read same value.
- ☐ Perfect coalescing, no conflicts.
- ☐ Each thread hits a different bank, no conflicts.

Correct answer

- ☒ Each thread hits a different bank, no conflicts.

Which CUDA API is used to transfer data from CPU to GPU memory?

1/1

- ☒ `cudaMemcpy(..., cudaMemcpyHostToDevice)`
- ☐ `cudaDeviceSynchronize()`
- ☐ `cudaMalloc()`
- ☐ `cudaMemcpy(..., cudaMemcpyDeviceToHost)`

Which of the following are correct regarding loop unrolling inside GPU kernels ? 0/1

- ☐ Loop unrolling may reduce occupancy (i.e., lesser number of active threads on an SM).
- ☐ Loop unrolling guarantees coalesced memory accesses by aligning loads.
- ☐ Loop unrolling guarantees better performance regardless of hardware constraints.
- ☒ Loop unrolling may expose more ILP.

Correct answer

- ☒ Loop unrolling may expose more ILP.
- ☒ Loop unrolling may reduce occupancy (i.e., lesser number of active threads on an SM).



```
__global__ void k(int* a)
{
    a[threadIdx.x] = threadIdx.x;
}
int main(){
    int *d;
    cudaMalloc(&d, 1024*sizeof(int));
    k<<<2,512>>>(d);
}
```

1/1

Which is correct?

- ☒ Writes at 0..1023 exactly once
- ☐ Writes only at indices 0..511
- ☐ Writes at 0..511 in each block, so the values at indices 0..511 incremented twice
- ☐ Undefined due to missing grid index

The CUDA kernel launch syntax mykernel<<<X, Y>>>() means:

1/1

- ☒ X blocks, Y threads per block
- ☐ Y blocks, X threads per block
- ☐ X blocks, each containing Y warps
- ☐ X grids, each with Y blocks

Shared memory in CUDA is:

1/1

- ☒ Faster than global memory
- ☒ Banked for parallel access
- ☒ Shared across all threads within a block
- ☐ Faster than L1 Cache



Which access patterns lead to coalescing?

0/1

- ☒ Thread 0 accesses $a[0]$, Thread 1 $\rightarrow a[1]$, ...
- ☐ Thread i accesses $a[\text{warp_id} + \text{threadIdx.x}]$
- ☐ Thread i accesses $a[\text{random}(i)]$
- ☐ Thread i accesses $a[2*i]$

Correct answer

- ☒ Thread 0 accesses $a[0]$, Thread 1 $\rightarrow a[1]$, ...
- ☒ Thread i accesses $a[\text{warp_id} + \text{threadIdx.x}]$

Which of the following GPU memories are *read-only* ?

1/1

- ☒ Constant Memory
- ☐ Global Memory
- ☒ Texture Memory
- ☐ Shared Memory

This form was created inside of IIT Hyderabad. - [Contact form owner](#)

Google Forms



